

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	CH. Durga Prasad	Reg. No.:	192425305
Section / Batch:	B.Tech/ AIML	Date:	05/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Q8. Case Study: Library Management search System(Binary Search) A digital library stores books in a sorted Catalog and needs fast retrieval of records. Analyze how Binary Search can be used to efficiently locate a Book. Identify requirements such as sorted data and quick access. Apply divide and conquer concepts to Explain the process. Analyze time complexity and performance. Design a solution and justify why Binary Search is appropriate

Code of Conduct

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Title

Library Management Search System Using Binary Search (Divide and Conquer Algorithm)

Description of the Real-World Problem

Modern libraries maintain thousands or even millions of book records in digital databases. Users expect quick and accurate search results when searching for a book using its Book ID, ISBN, or Title. If the system searches every record one by one, the response time becomes slow as the number of books increases.

To improve efficiency, the library stores book records in sorted order based on a unique key such as Book ID or ISBN. Binary Search uses the Divide and Conquer strategy to repeatedly divide the search space into two halves until the required book is found. This significantly reduces the search time and improves the overall performance of the library management system.

Problem Statement

Design an efficient searching mechanism for a digital library management system that quickly locates a book from a sorted collection of records. The system should minimize search time, improve user experience, and efficiently handle large datasets by implementing the Binary Search algorithm based on the Divide and Conquer technique.

Algorithm Used

Binary Search Algorithm (Divide and Conquer)

Binary Search works only on sorted data.

Algorithm Steps

1. Store all book records in sorted order.
2. Initialize two pointers:
 - Low = 0
 - High = Number of books - 1

3. Calculate the middle index.

$$\text{Mid} = (\text{Low} + \text{High}) // 2$$

4. Compare the target Book ID with the middle element.
5. If equal, return the record.
6. If target is smaller, search the left half.
7. If target is greater, search the right half.

8. Repeat until the element is found or $Low > High$.
9. If not found, display "Book Not Available."

Justification for Choosing the Algorithm (Why Binary Search?)

Binary Search is an ideal algorithm for library management systems because:

- Library records are generally maintained in sorted order.
- It significantly reduces search time compared to Linear Search.
- It follows the Divide and Conquer strategy, making it highly efficient.
- Suitable for large databases containing thousands of books.
- Minimizes unnecessary comparisons.
- Provides quick response to users.
- Easy to implement and maintain.

Complexity Analysis

Case	Time Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Space Complexity (Iterative) $O(1)$

Explanation

- Best Case: Book is found at the middle position.
- Average Case: Search space is repeatedly divided into halves.
- Worst Case: Continues dividing until one element remains.
- Space complexity is constant in the iterative approach.

Manual Execution (Step-by-Step Process)

Sorted Book IDs

[101, 105, 110, 115, 120, 125, 130, 135, 140]

Search Book ID = 125

Step 1

Low = 0

High = 8

Mid = $(0 + 8) // 2 = 4$

Middle Element = 120

101 105 110 115 [120] 125 130 135 140

Since

$125 > 120$

Search Right Half

Step 2

Low = 5

High = 8

Mid = $(5 + 8) // 2 = 6$

Middle Element = 130

125 [130] 135 140

Since

$125 < 130$

Search Left Half

Step 3

Low = 5

High = 5

Mid = 5

Middle Element = 125

Book Found

Search completed successfully.

Applications

Binary Search is widely used in:

- Digital Library Management Systems
- Student Record Management Systems
- Employee Database Systems
- Online Shopping Product Search
- Banking Record Search
- Hospital Patient Record Systems
- Dictionary and Contact Search Applications
- Search Engines
- File Management Systems
- Database Index Searching

Advantages

- Very fast searching algorithm.
- Efficient for large datasets.
- Requires fewer comparisons.
- Reduces execution time.
- Uses Divide and Conquer strategy.
- Easy to implement.
- Low memory usage (iterative version).
- Improves system performance.

Limitations

- Works only on sorted data.
- Sorting is required before searching.
- Not suitable for frequently changing unsorted data.
- Random access to data is required.

- Less effective for linked lists.

Comparison with Other Algorithms

Feature	Linear Search	Binary Search
Data Requirement	Unsorted or Sorted	Sorted Only
Technique	Sequential Search	Divide and Conquer
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Suitable for Large Data	No	Yes
Efficiency	Lower	Higher

Python Program

```
def binary_search(book_ids, target):
```

```
    low = 0
```

```
    high = len(book_ids) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if book_ids[mid] == target:
```

```
            return mid
```

```
        elif book_ids[mid] < target:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1
```

```
books = [101, 105, 110, 115, 120, 125, 130, 135, 140]
```

```
print("Available Book IDs:")

print(books)

target = int(input("\nEnter Book ID to search: "))

result = binary_search(books, target)

if result != -1:

    print("\nBook Found!")

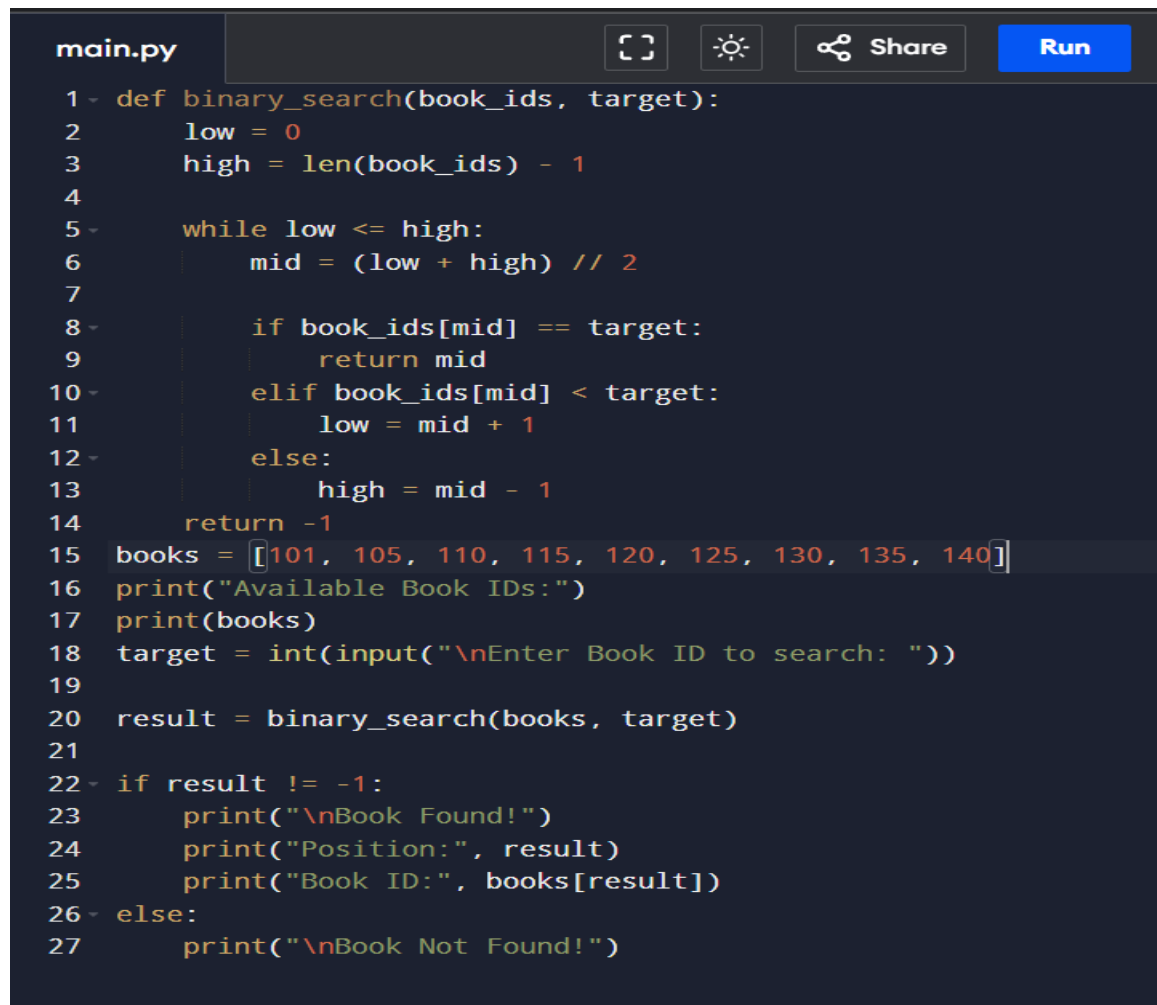
    print("Position:", result)

    print("Book ID:", books[result])

else:

    print("\nBook Not Found!")
```

Screenshot of the Program

A screenshot of a code editor window titled 'main.py'. The editor has a dark theme and includes icons for full screen, settings, and sharing, along with a blue 'Run' button. The code is a Python program that implements a binary search algorithm. It defines a function 'binary_search' that takes a list of book IDs and a target ID as input. The function returns the index of the target ID if found, or -1 if not found. The main part of the program initializes a list 'books' with values [101, 105, 110, 115, 120, 125, 130, 135, 140], prints the list, prompts the user for a target ID, and then calls the 'binary_search' function. Depending on the result, it prints either the book ID found at the specified position or a message stating the book was not found.

```
main.py  [Full Screen] [Settings] [Share] [Run]

1 def binary_search(book_ids, target):
2     low = 0
3     high = len(book_ids) - 1
4
5     while low <= high:
6         mid = (low + high) // 2
7
8         if book_ids[mid] == target:
9             return mid
10        elif book_ids[mid] < target:
11            low = mid + 1
12        else:
13            high = mid - 1
14    return -1
15 books = [101, 105, 110, 115, 120, 125, 130, 135, 140]
16 print("Available Book IDs:")
17 print(books)
18 target = int(input("\nEnter Book ID to search: "))
19
20 result = binary_search(books, target)
21
22 if result != -1:
23     print("\nBook Found!")
24     print("Position:", result)
25     print("Book ID:", books[result])
26 else:
27     print("\nBook Not Found!")
```

Output Screenshot

Sample Output 1

Available Book IDs:

[101, 105, 110, 115, 120, 125, 130, 135, 140]

Enter Book ID to search: 125

Book Found!

Position: 5

```
Output
Available Book IDs:
[101, 105, 110, 115, 120, 125, 130, 135, 140]

Enter Book ID to search: 120

Book Found!
Position: 4
Book ID: 120

=== Code Execution Successful ===
```

Conclusion

Binary Search is one of the most efficient searching algorithms for library management systems where book records are stored in sorted order. By applying the Divide and Conquer approach, it reduces the search space by half during each iteration, resulting in a time complexity of $O(\log n)$. Compared with Linear Search, Binary Search provides much faster retrieval of book records, making it highly suitable for digital libraries containing large collections of books. Its simplicity, efficiency, and scalability make it an excellent choice for implementing fast and reliable search functionality in modern library management systems.